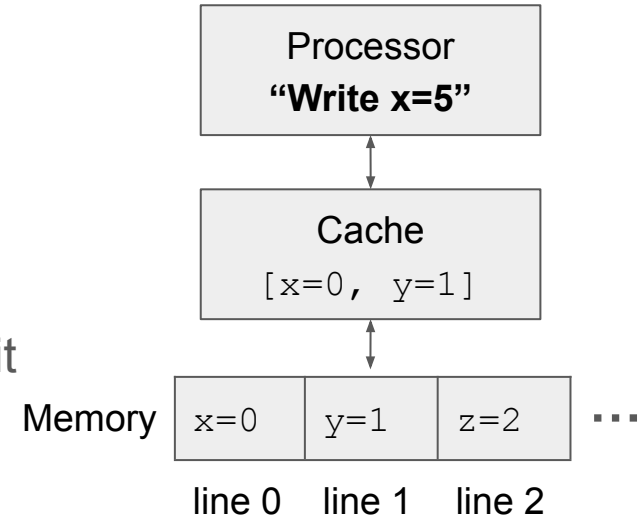
Cache Coherence

Guest lecturer: Marcin Anforowicz

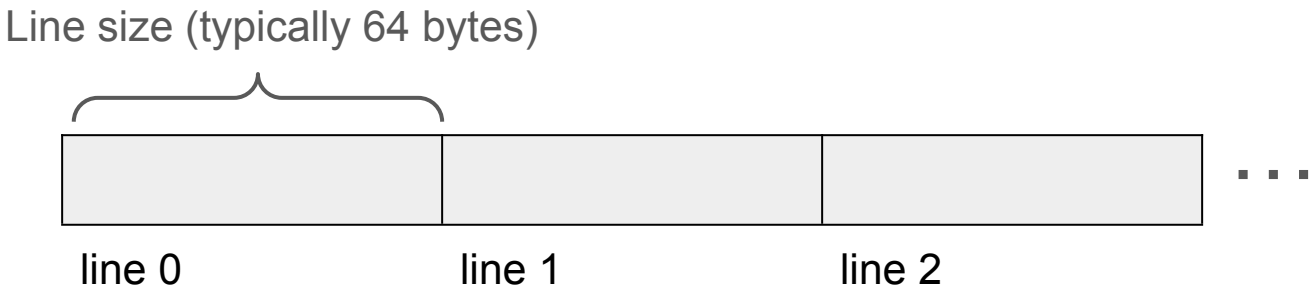
Caching

- Main memory is slow, so we store copies of recently-used *cache lines* in the cache.
- The cache is fast because it's small, near the processor, and uses SRAM instead of DRAM.
- Cache is "transparent" - the processor can think it reads and writes directly to memory.

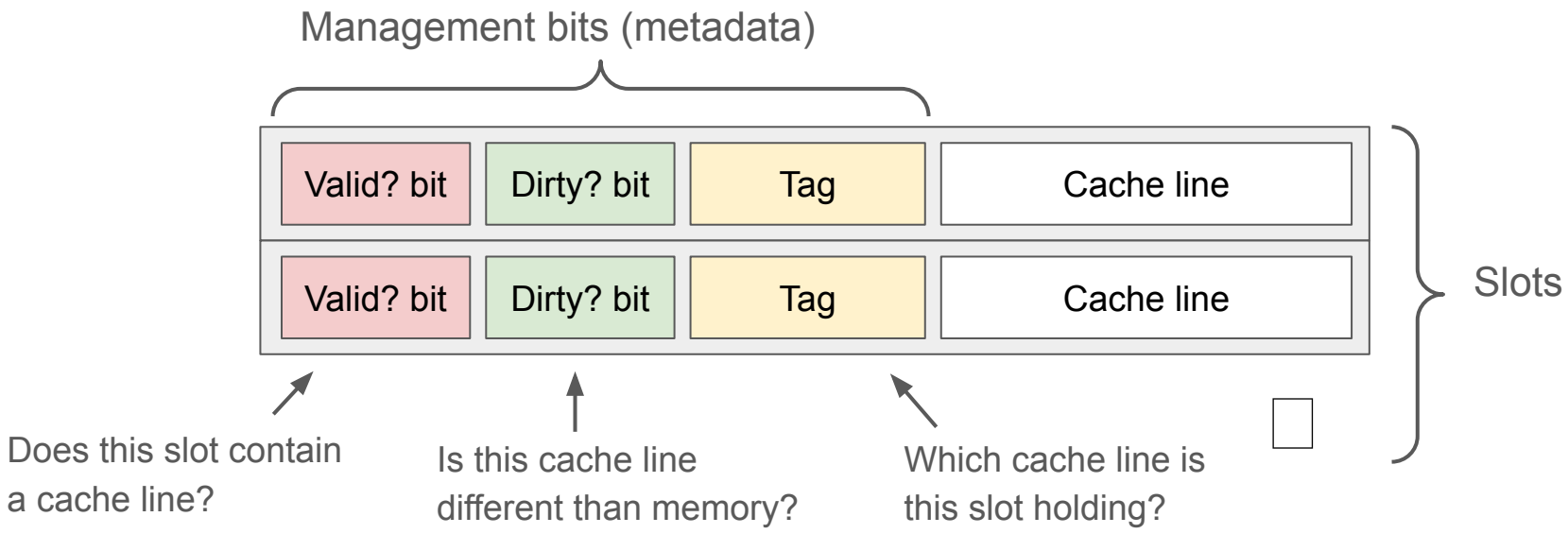


Memory

- For caching, memory is divided into **“cache lines”** (common term in literature).
- (Sometimes called “blocks”, like in CSE 351)



Cache

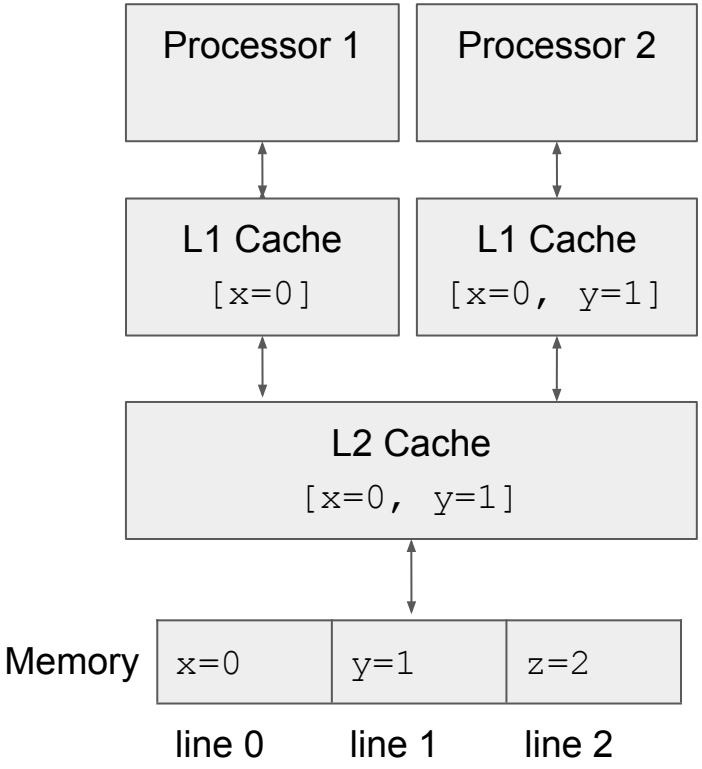

Management bits (metadata)
Valid? bit
Dirty? bit
Tag
Cache line
Valid? bit
Dirty? bit
Tag
Cache line
Slots
Does this slot contain a cache line?
Is this cache line different than memory?
Which cache line is this slot holding?

Typical cache policies

- on write-hit: **write-back** (instead of **write-through**).
  - Modify the line only in the cache. Write-back to memory later, when it's evicted/invalidated.
- on write-miss: **write-allocate** (instead of **no write-allocate**).
  - Fetch the line from memory into the cache before writing to it.

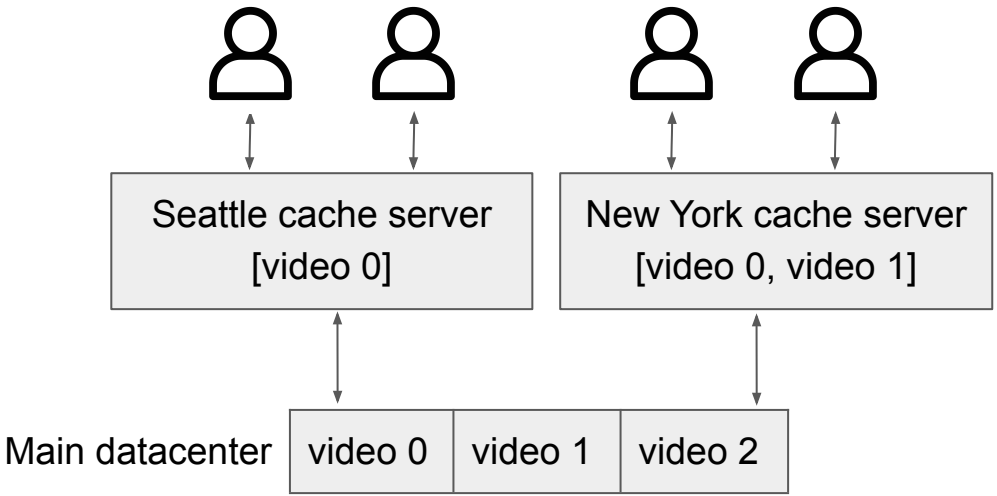
Multi-core

- Each core has its own cache, for performance.
- The same line might be stored in multiple caches.

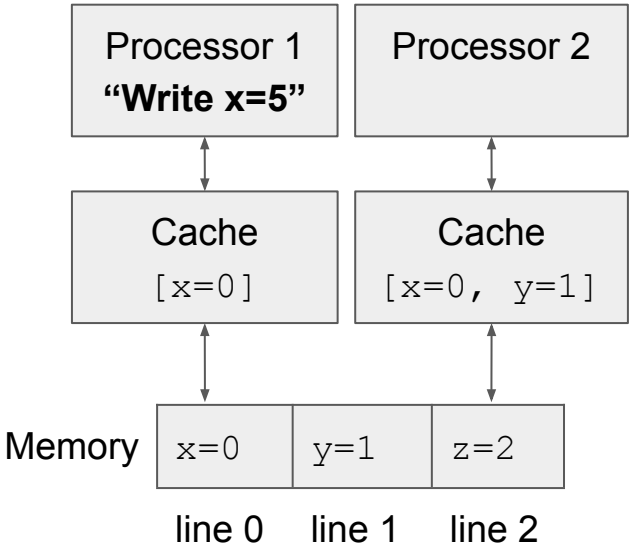


Higher level use of caching

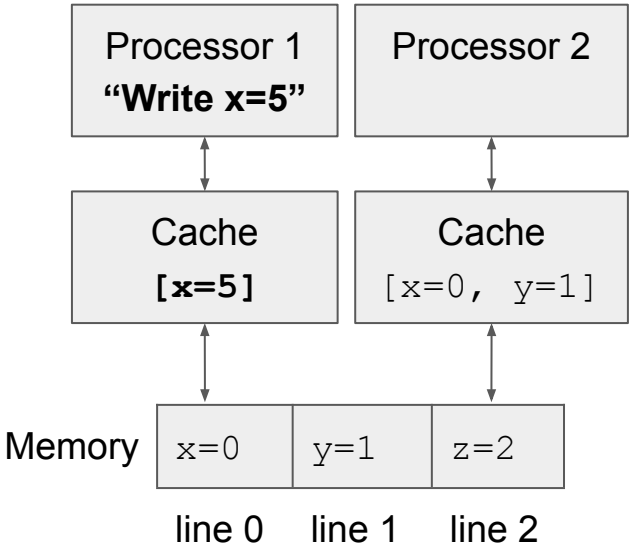
CDN (Content Distribution Network)



Naive scenario

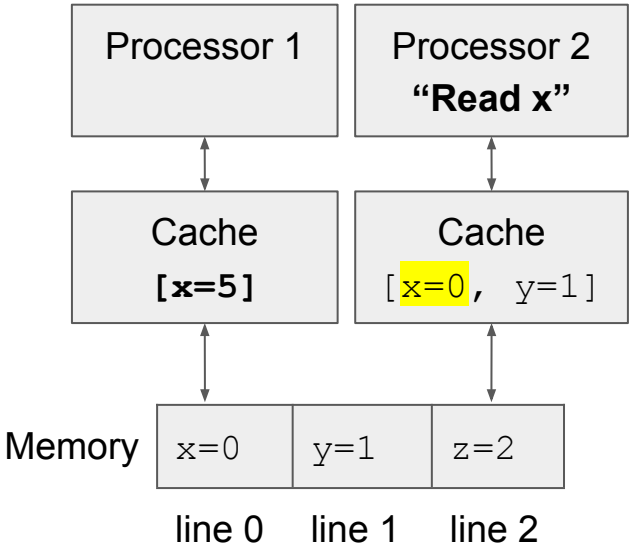


Naive scenario



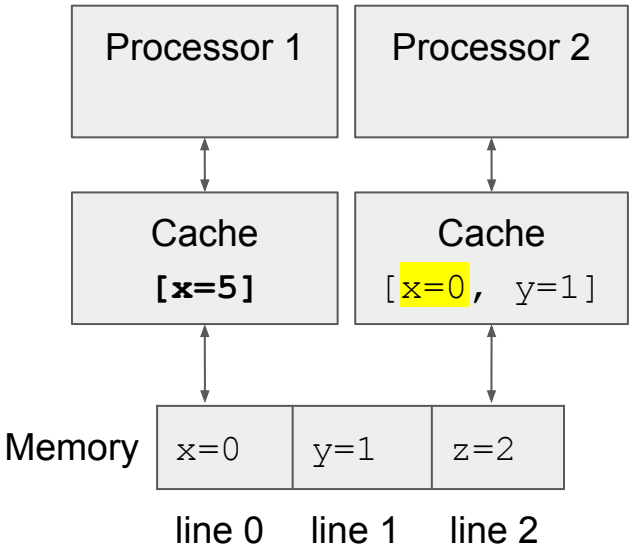
Naive scenario

- Processors disagree about x.
- Synchronization and atomics don't solve this.
- Thankfully, cache coherence fixes this!



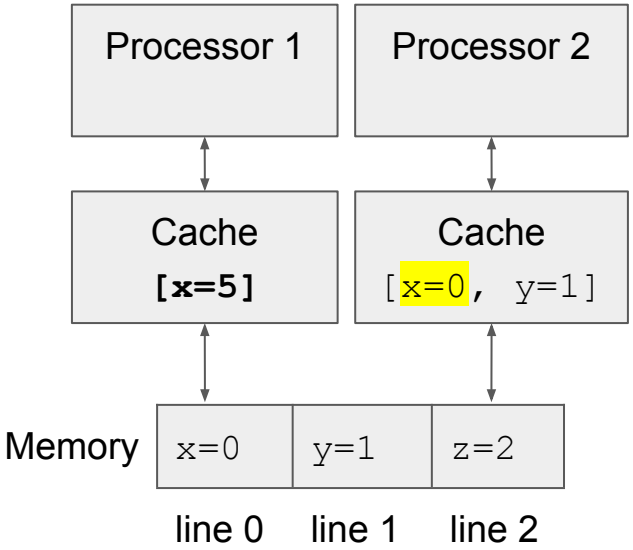
Goal of cache coherence

- Make caching “transparent”: programmer can think they access memory directly.
- Provide processors with some consistency model for each memory location.
- A typical consistency model:
  - Write propagation: Each write must eventually be seen by later reads from all processors.
  - Write serialization: For each memory location, there’s a global ordering of writes that agrees with what each processor saw.



MSI Protocol

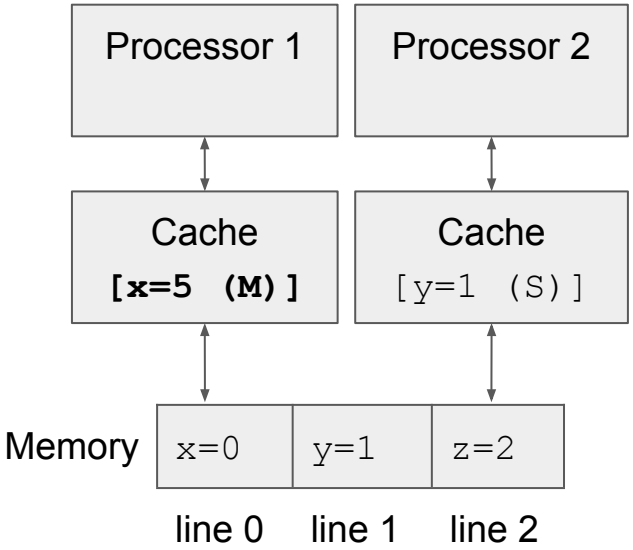
- **Goal:** If multiple caches have a copy of the same cache line, all copies must be the same.
- **Therefore:** A cache may only modify a cache line if no other cache has a copy of it.
- **To do this:** Each cache slot has metadata saying whether its line can be modified.



MSI Protocol

Cache slot:

- **Goal:** If multiple caches have a copy of the same cache line, all copies must be the same.
- **Therefore:** A cache may only modify a cache line if no other cache has a copy of it.
- **To do this:** Each cache slot has metadata saying whether its line can be modified.



MSI Protocol

Cache slot:

Each cache line is in one of three states:

- **M (Modified)**: Only this cache has this line. May be modified.
- **S (Shared)**: This line may exist in many caches. Read-only. Matches memory.
- **I (Invalid)**: This slot doesn't have a line.

❌ - not allowed
✅ - allowed

State of line in cache 1:

State of same line in cache 2:

	M	S	I
M	❌	❌	✅
S	❌	✅	✅
I	✅	✅	✅

How do we uphold MSI?

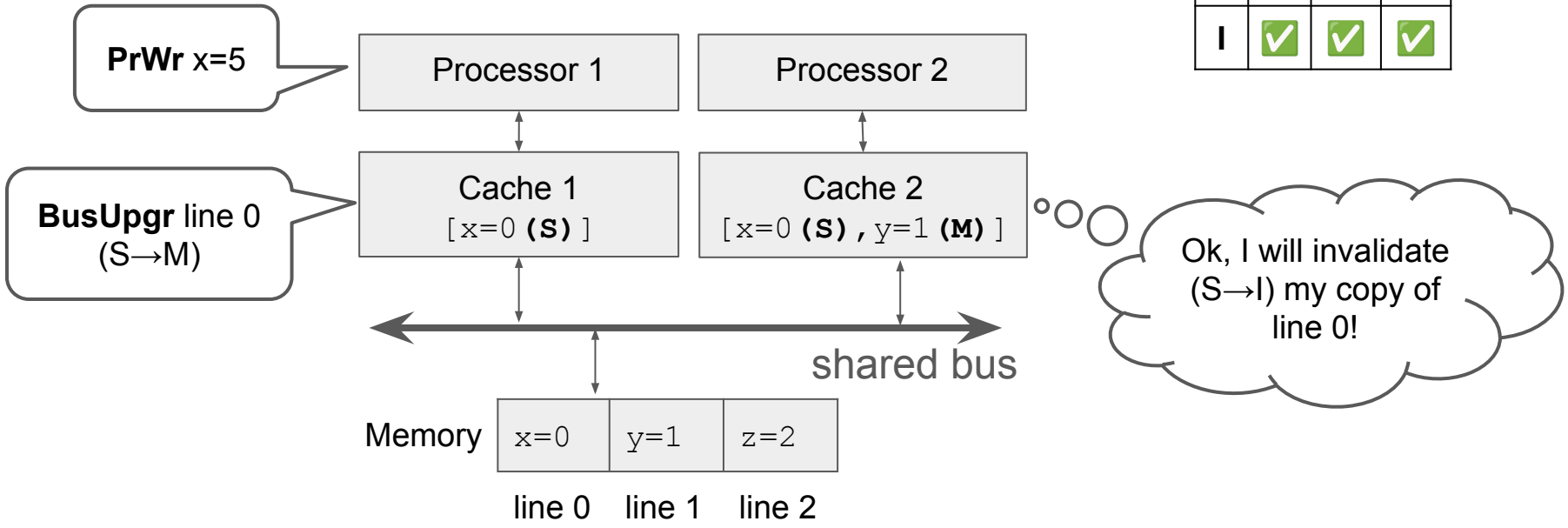
	M	S	I
M	❌	❌	✅
S	❌	✅	✅
I	✅	✅	✅

Option 1: Snooping

- Each cache listens to all memory requests made by other caches.
- If it hears another cache acquire a line, it downgrades or invalidates its own copy of that line, as needed to uphold MSI.

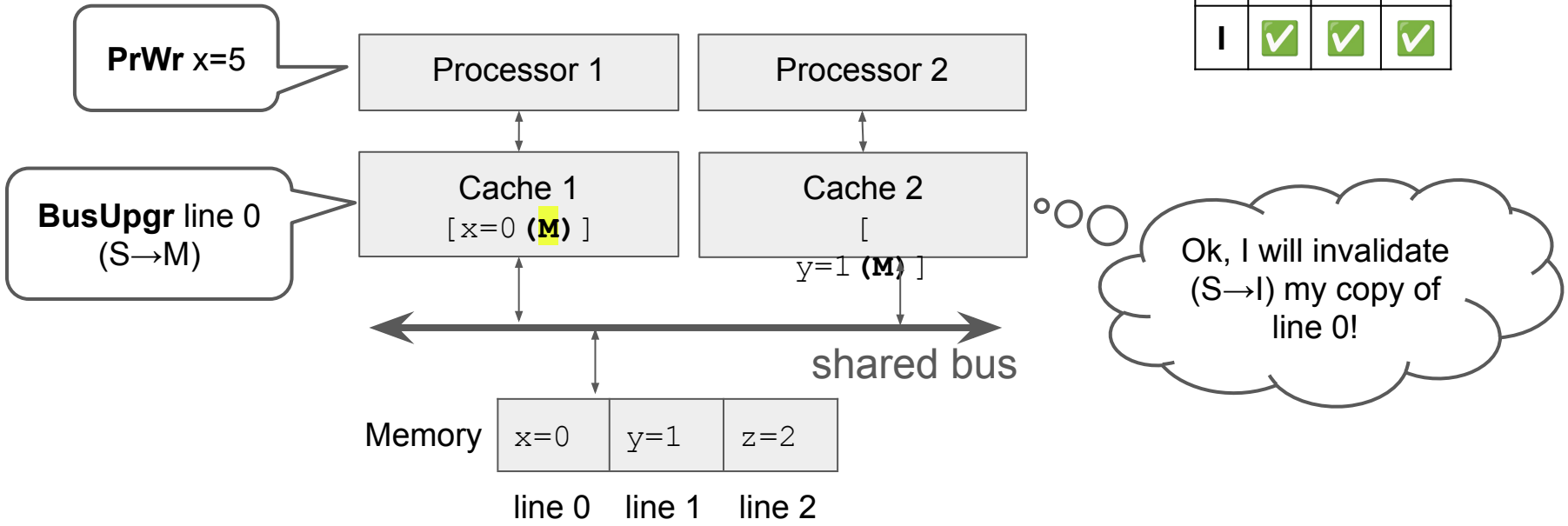
Snooping

	M	S	I
M	❌	❌	✅
S	❌	✅	✅
I	✅	✅	✅



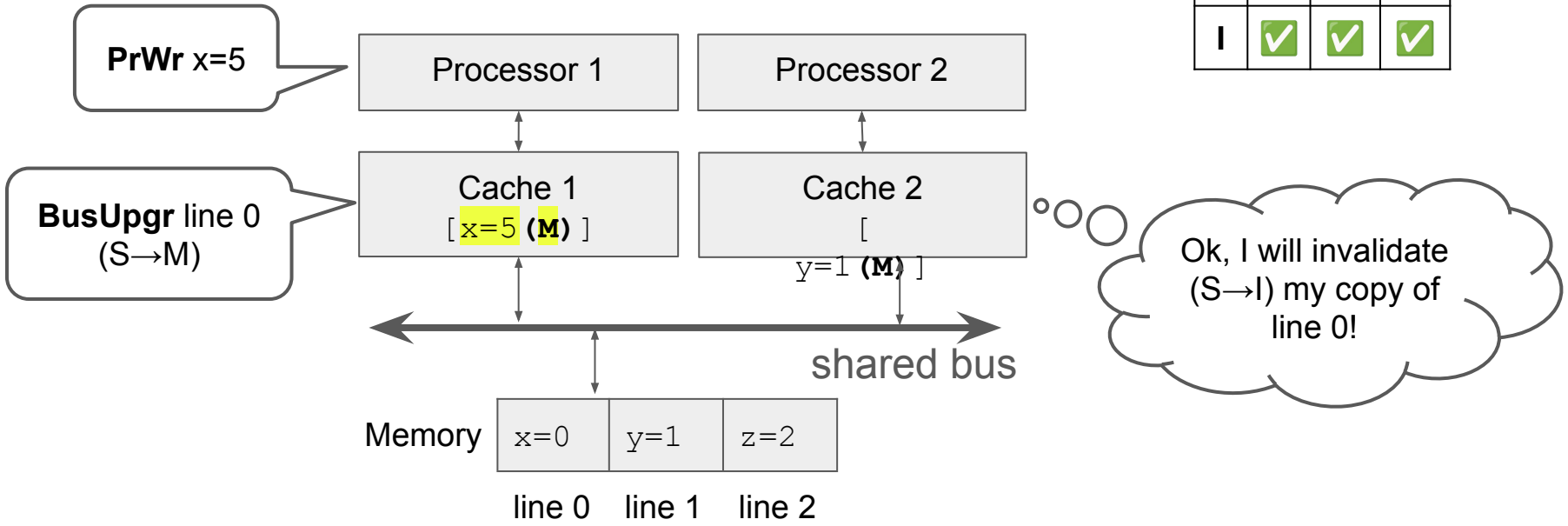
Snooping

	M	S	I
M	❌	❌	✅
S	❌	✅	✅
I	✅	✅	✅



Snooping

	M	S	I
M	❌	❌	✅
S	❌	✅	✅
I	✅	✅	✅



List of messages in snooping MSI

	M	S	I
M	❌	❌	✅
S	❌	✅	✅
I	✅	✅	✅

Processor→cache (direct)

- PrRd, PrWr — Processor requests read or write to an address.

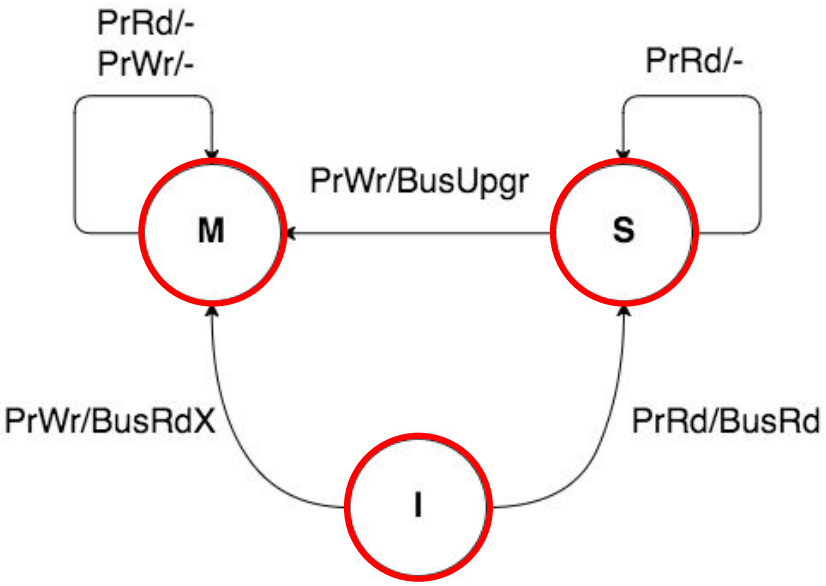
Cache→shared bus (seen by memory, and all caches)

- BusRd (I→S) Cache read from memory.
- BusRdX (I→M) Cache read *exclusive* from memory.
- BusUpgr (S→M) Cache tells everyone it's upgrading a line.
- Flush (M→I/S) Cache writes back a modified line.

Snooping MSI from cache's perspective

Cache line transitions on receiving processor requests.

Notation: *received / sending*

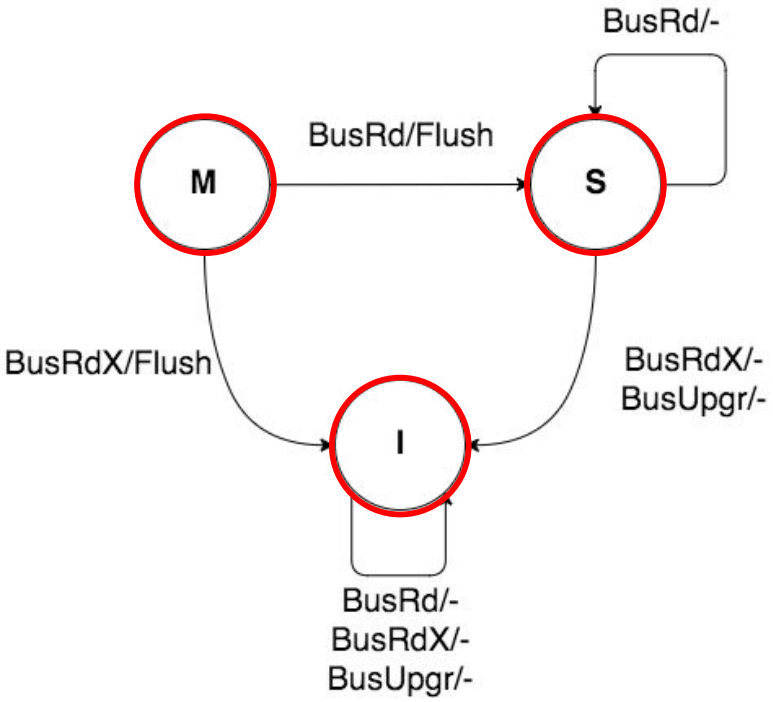


By Sandeep993 - Own work, CC BY-SA 4.0

Snooping MSI from cache's perspective

Cache line transitions on overhearing bus messages from other caches.

Notation: *received / sending*



By Sandeep993 - Own work, CC BY-SA 4.0

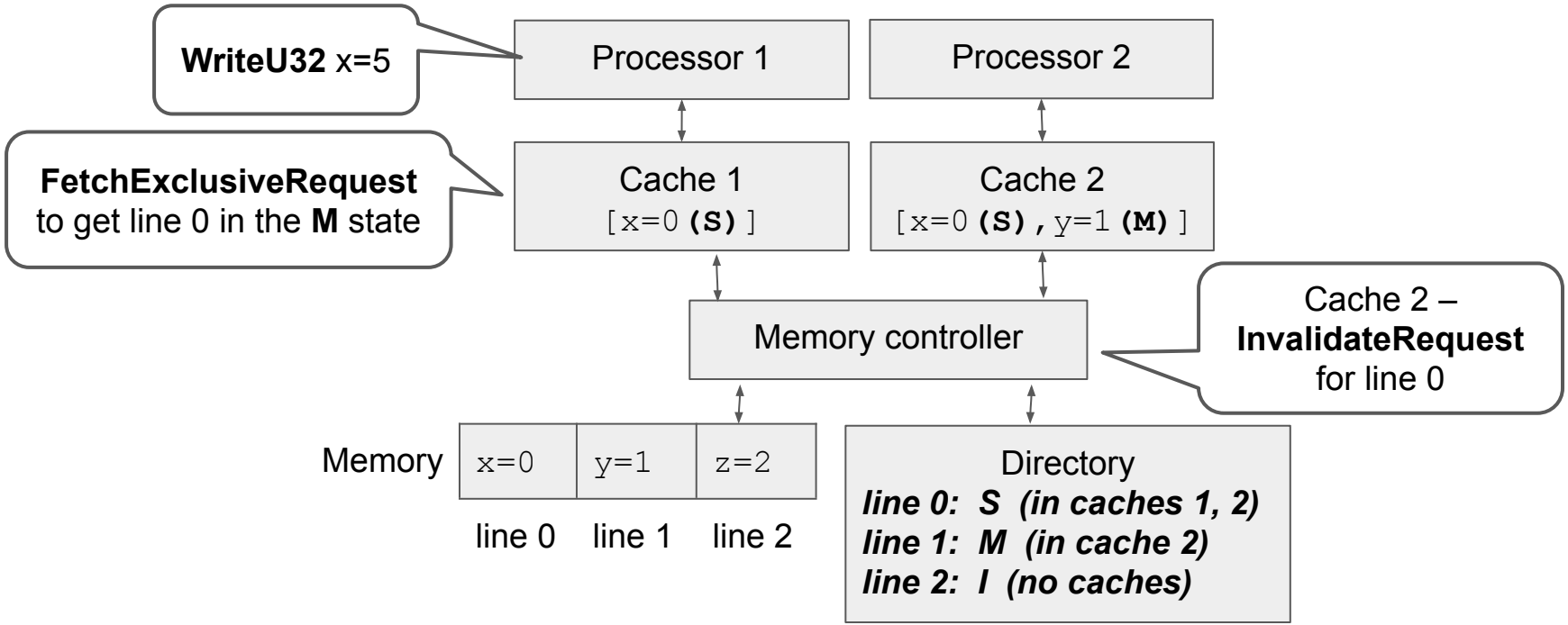
How do we uphold MSI?

	M	S	I
M	❌	❌	✅
S	❌	✅	✅
I	✅	✅	✅

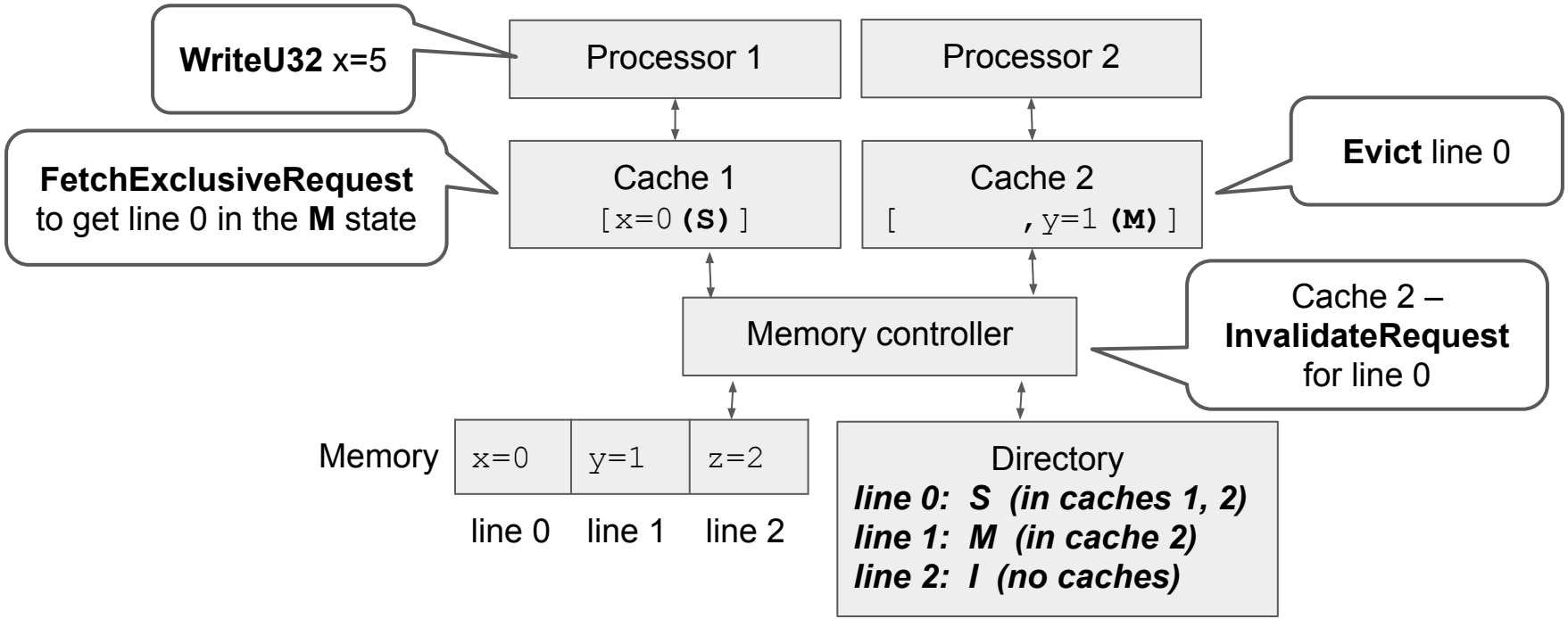
Option 2: Directory

- Cache sends transitions requests (like **I→M**) to a memory controller.
- Memory controller has a *directory* that tracks the states of lines in all caches.
- The memory controller sends invalidate requests to caches, to uphold MSI.

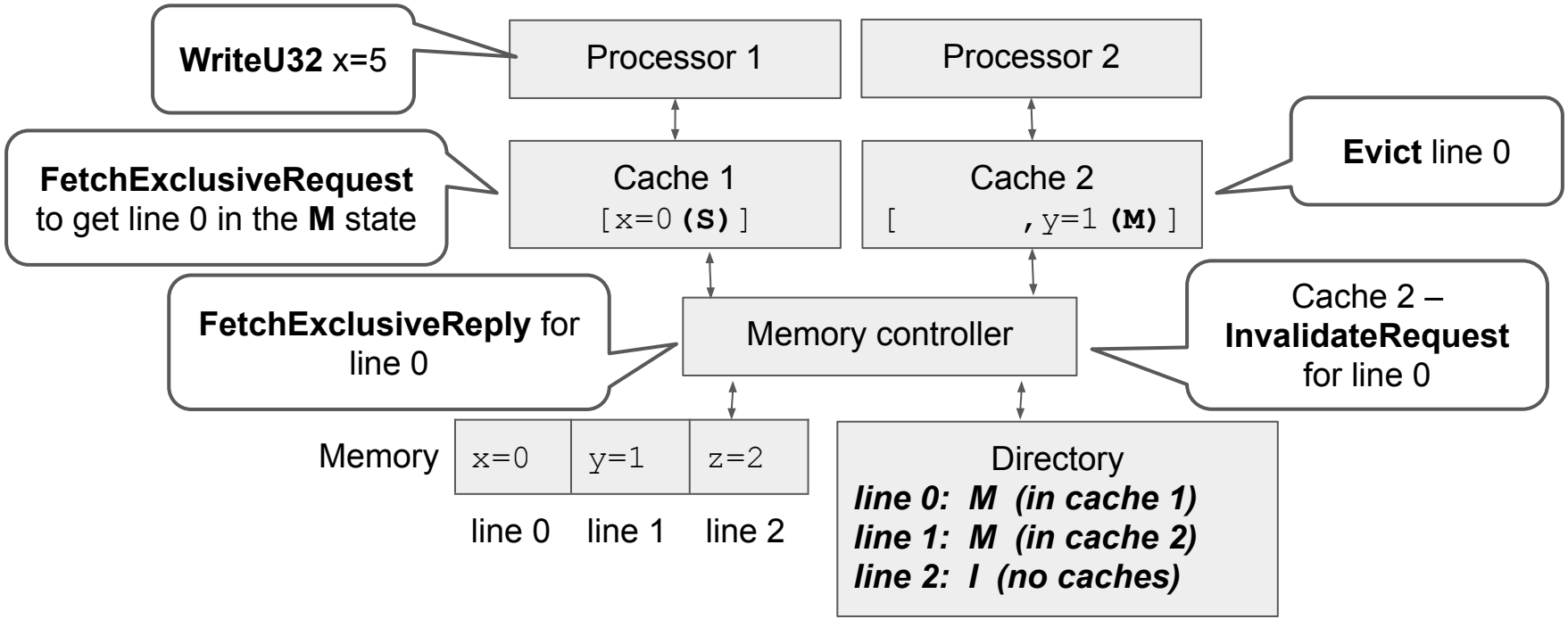
Directory-based



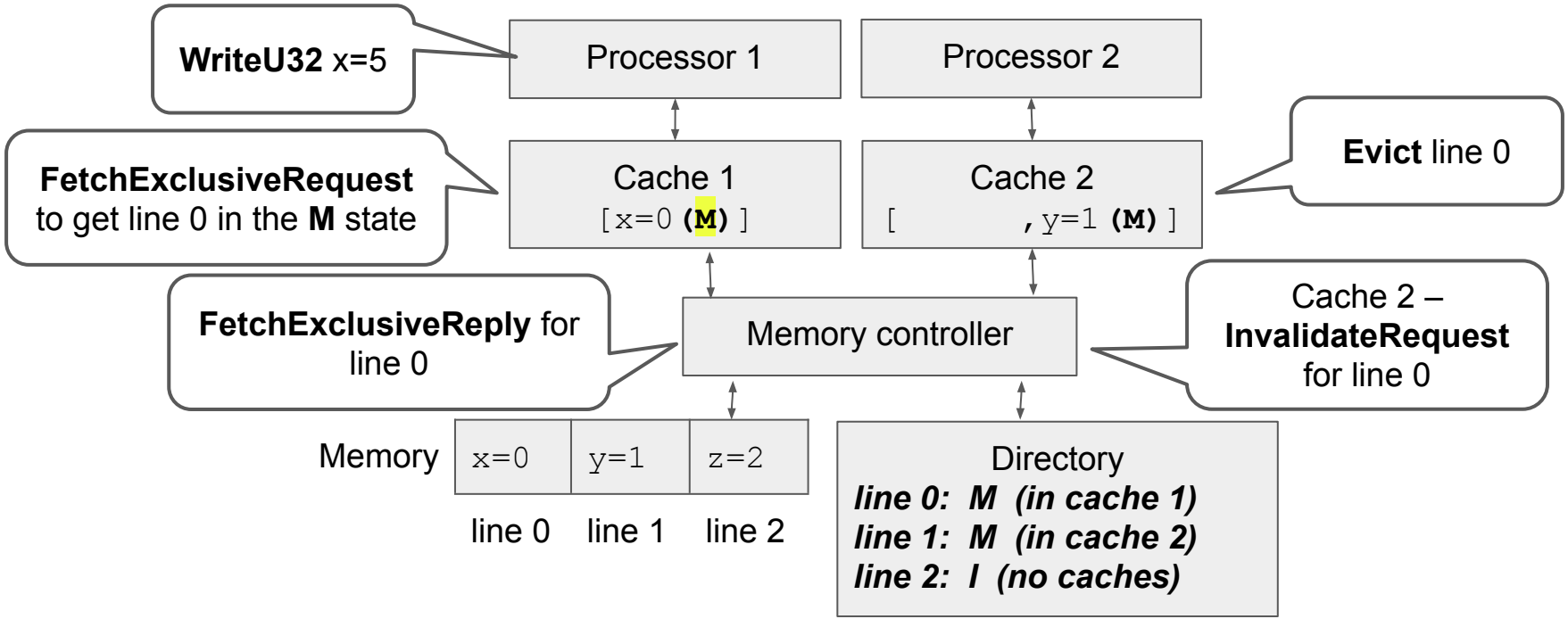
Directory-based



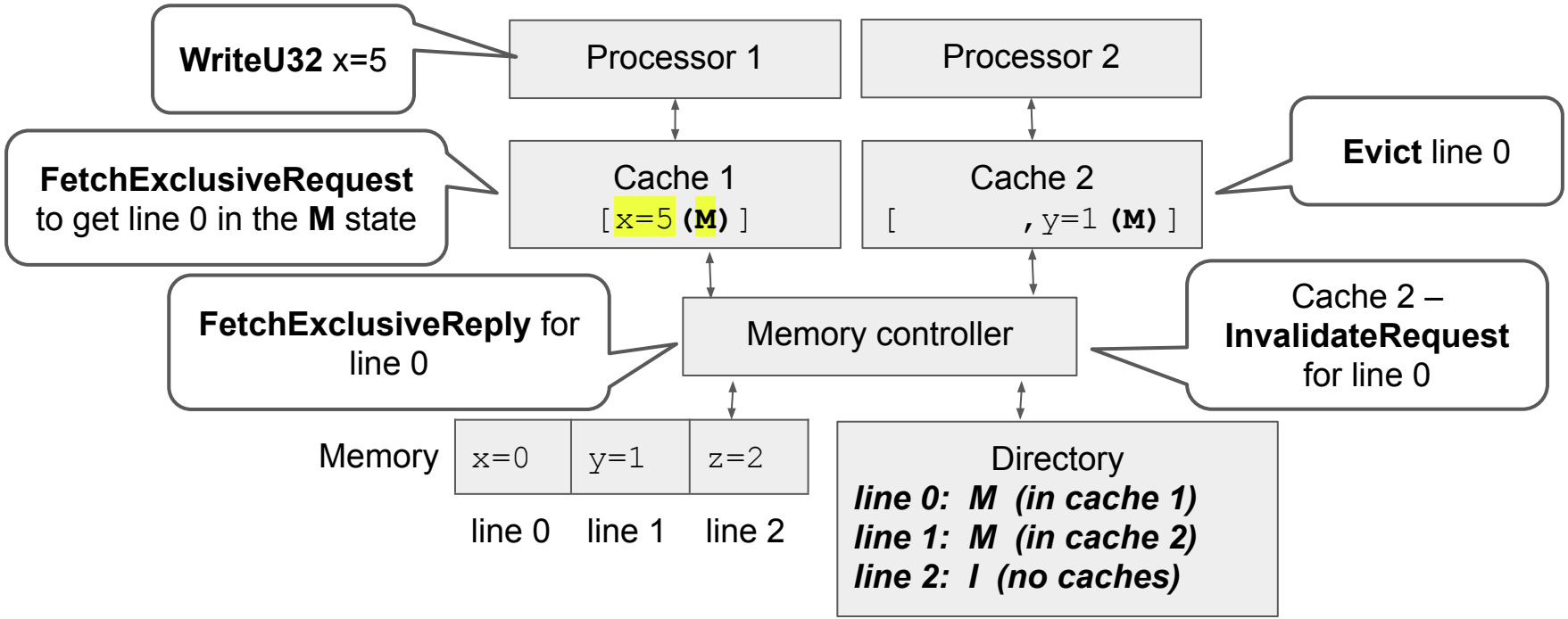
Directory-based



Directory-based



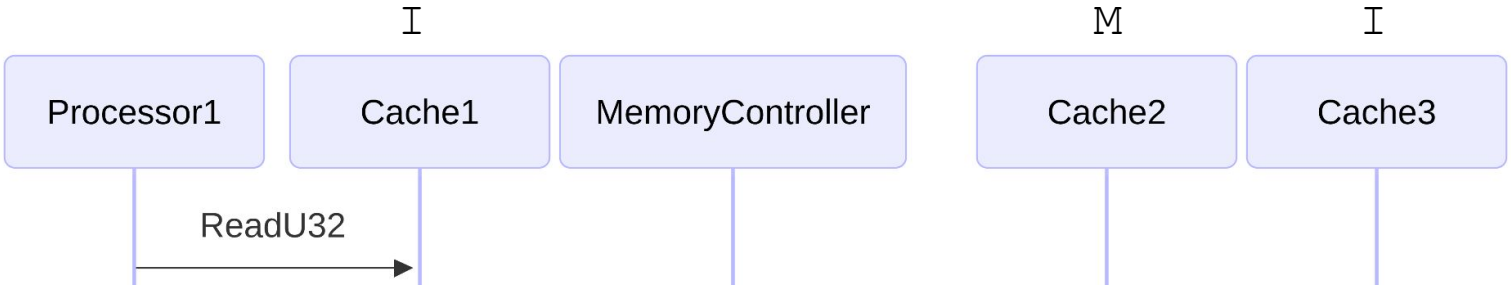
Directory-based



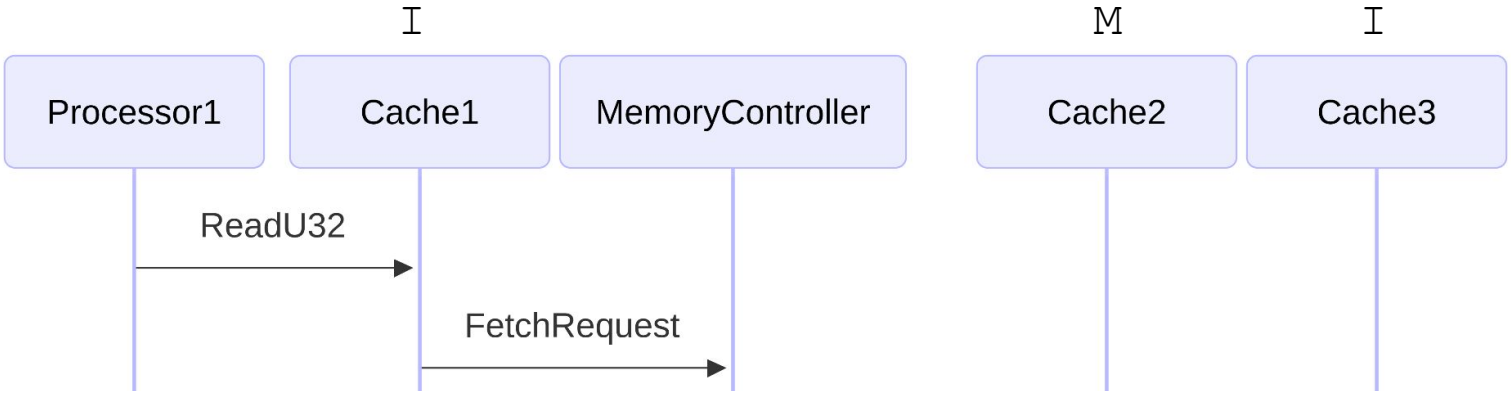
Directory-based MSI read example

(states shown for the cache line being accessed)

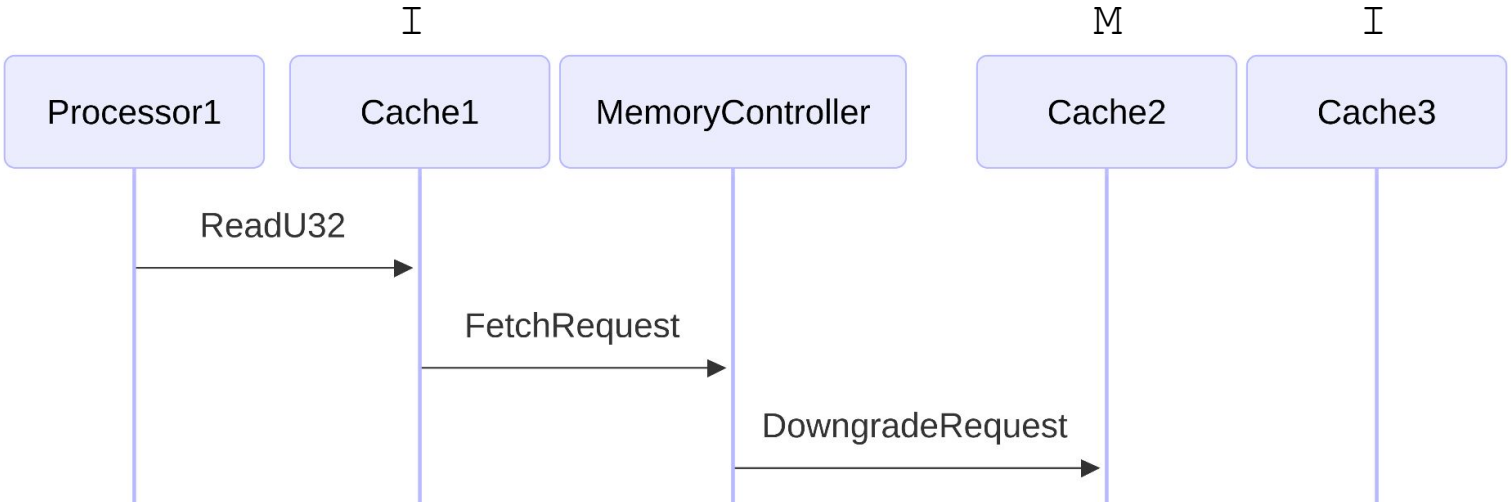
Directory-based MSI read example



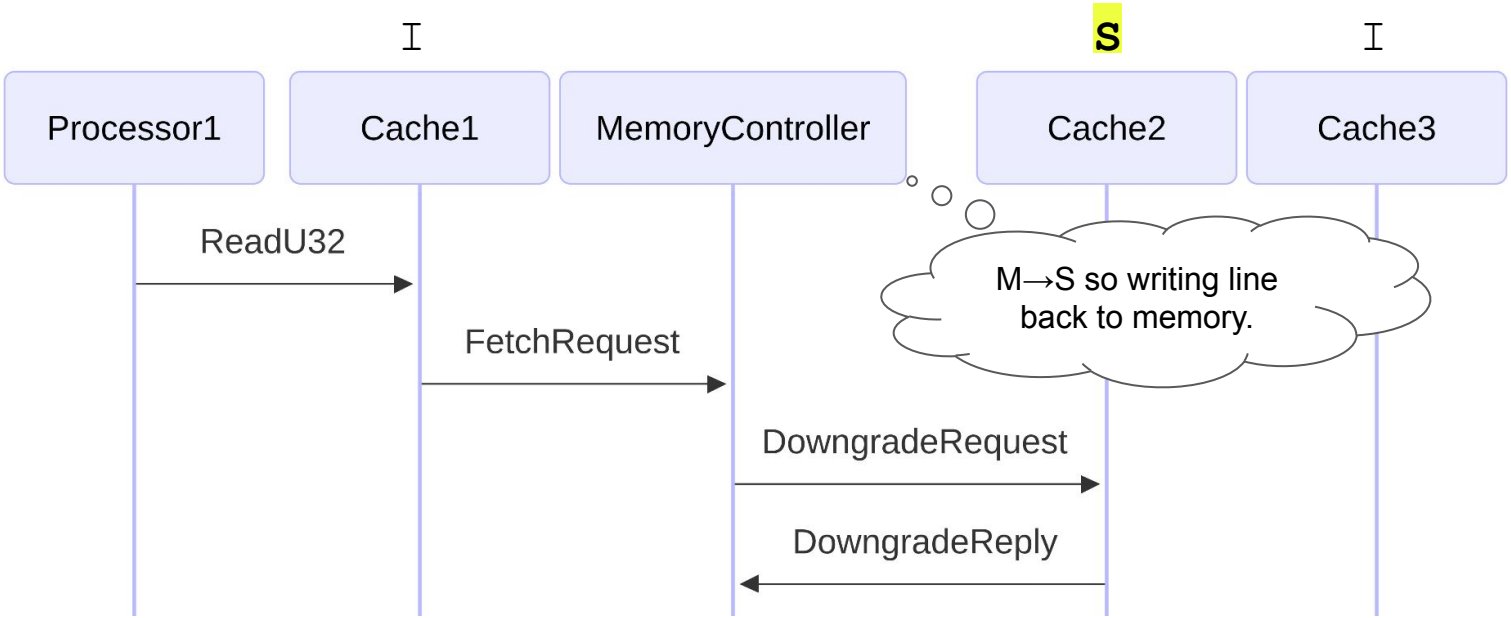
Directory-based MSI read example



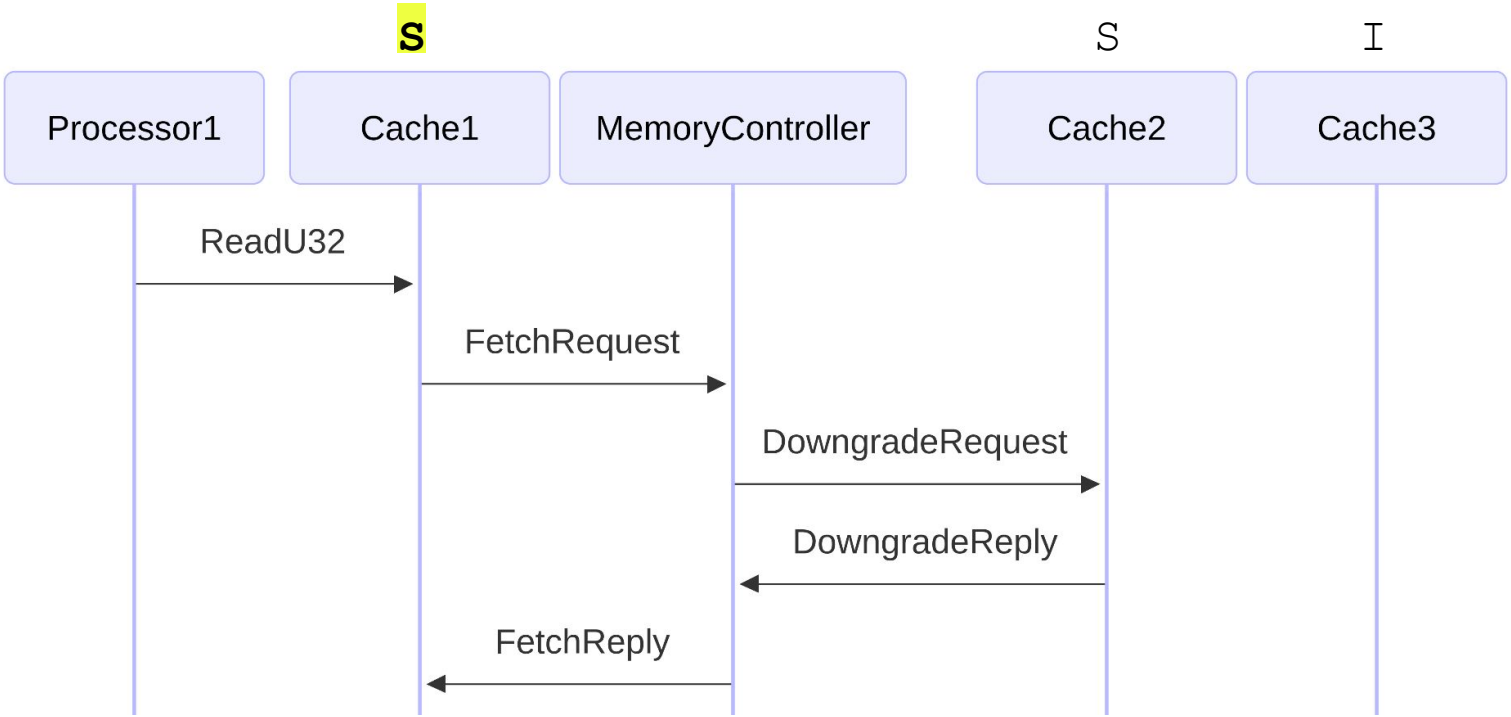
Directory-based MSI read example



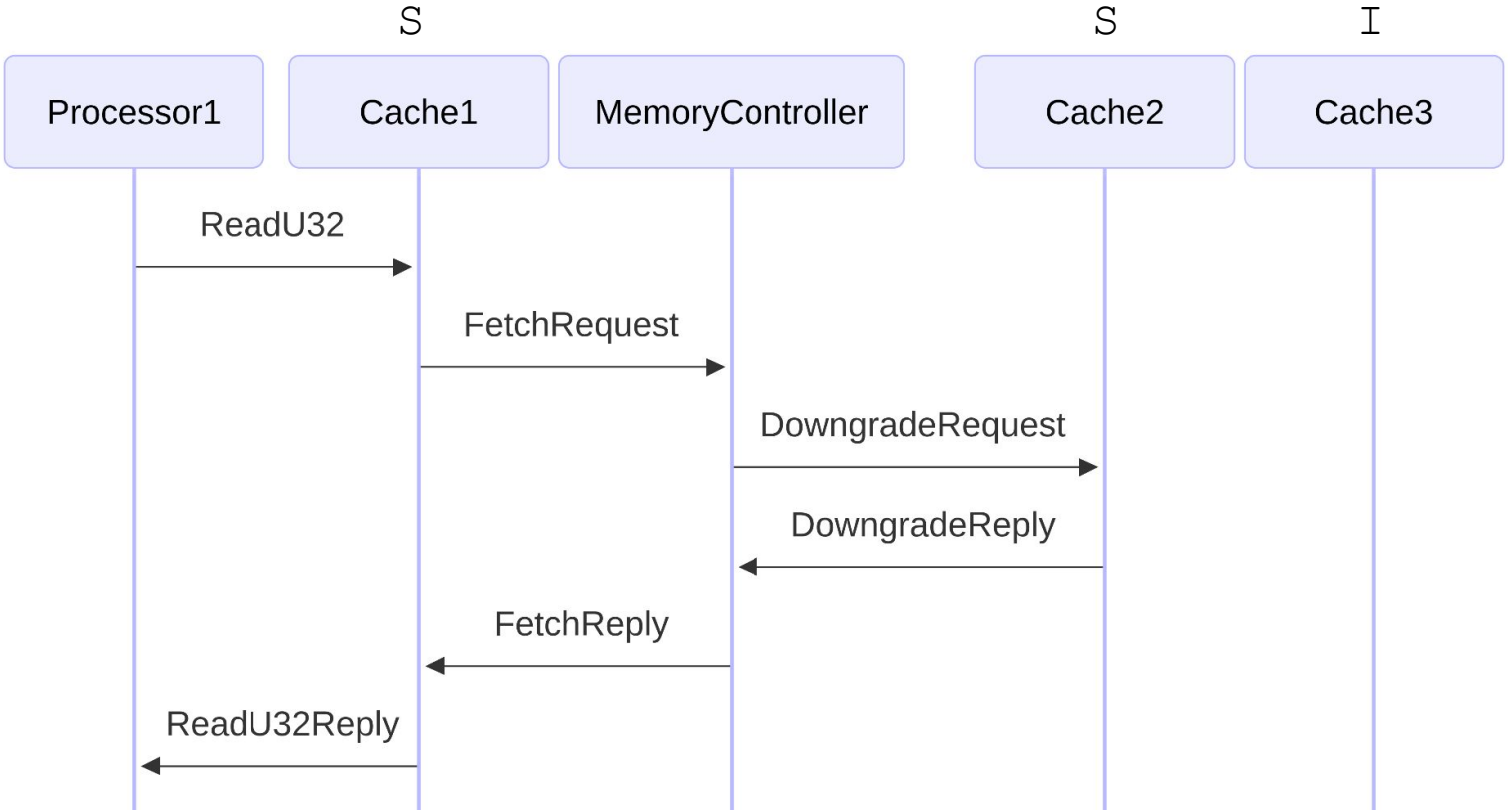
Directory-based MSI read example



Directory-based MSI read example

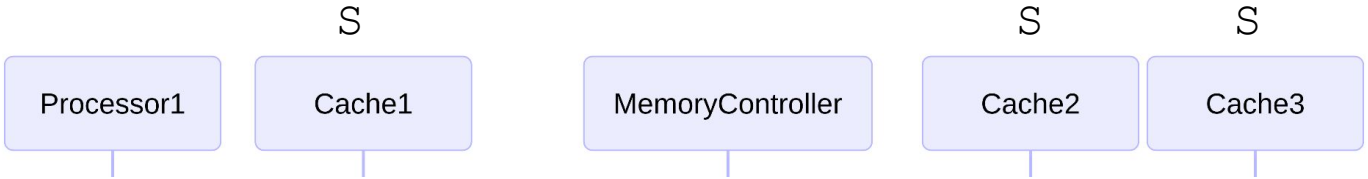


Directory-based MSI read example



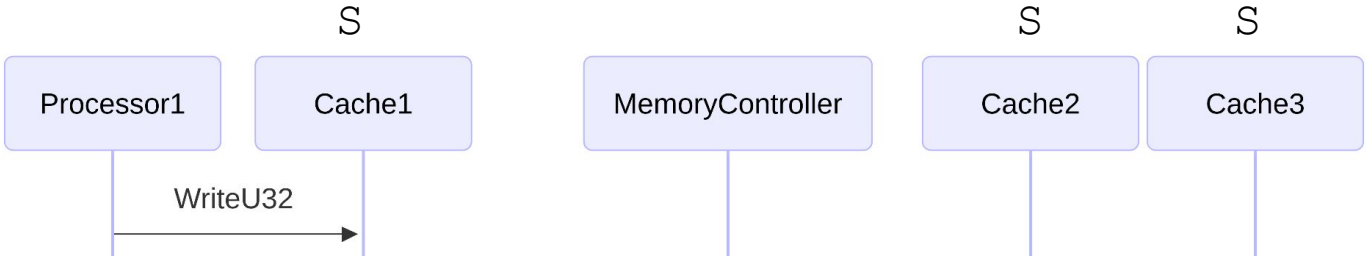
Directory-based MSI write example

(states shown for the cache line being accessed)

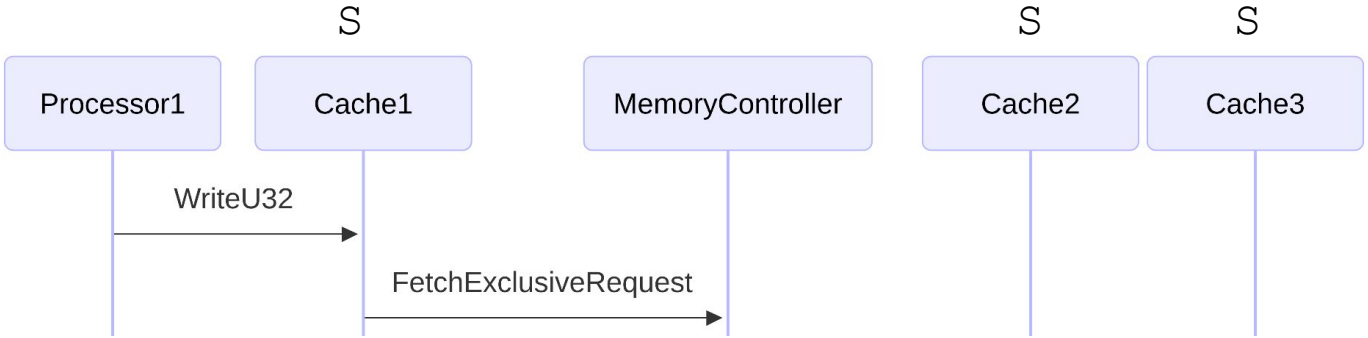


Suppose Processor1 just read a counter, and now it wants to write an incremented value.

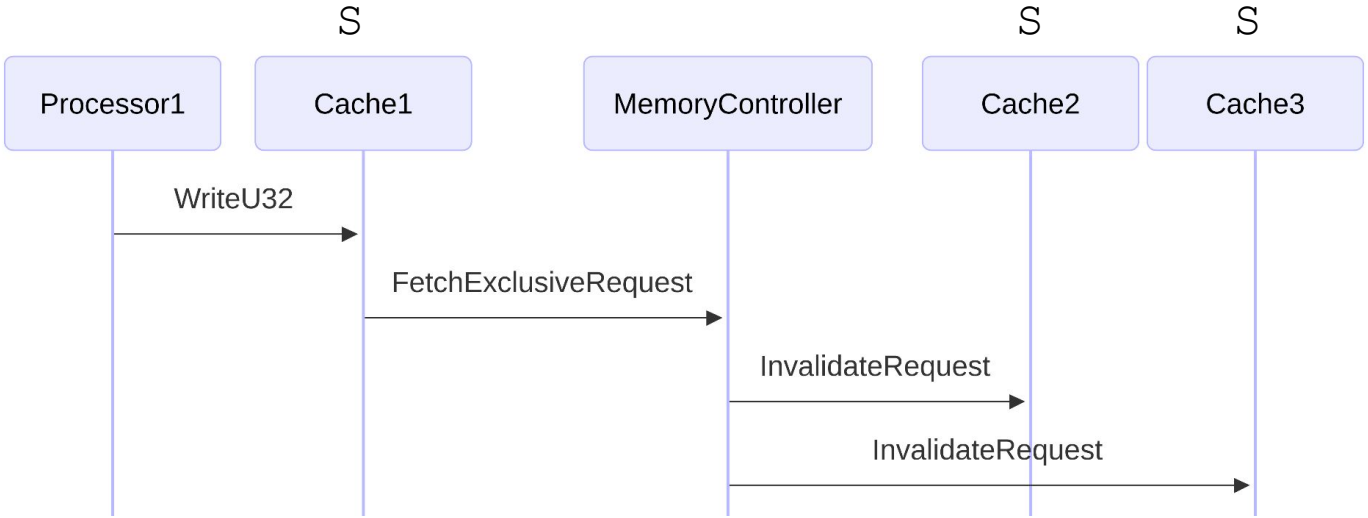
Directory-based MSI write example



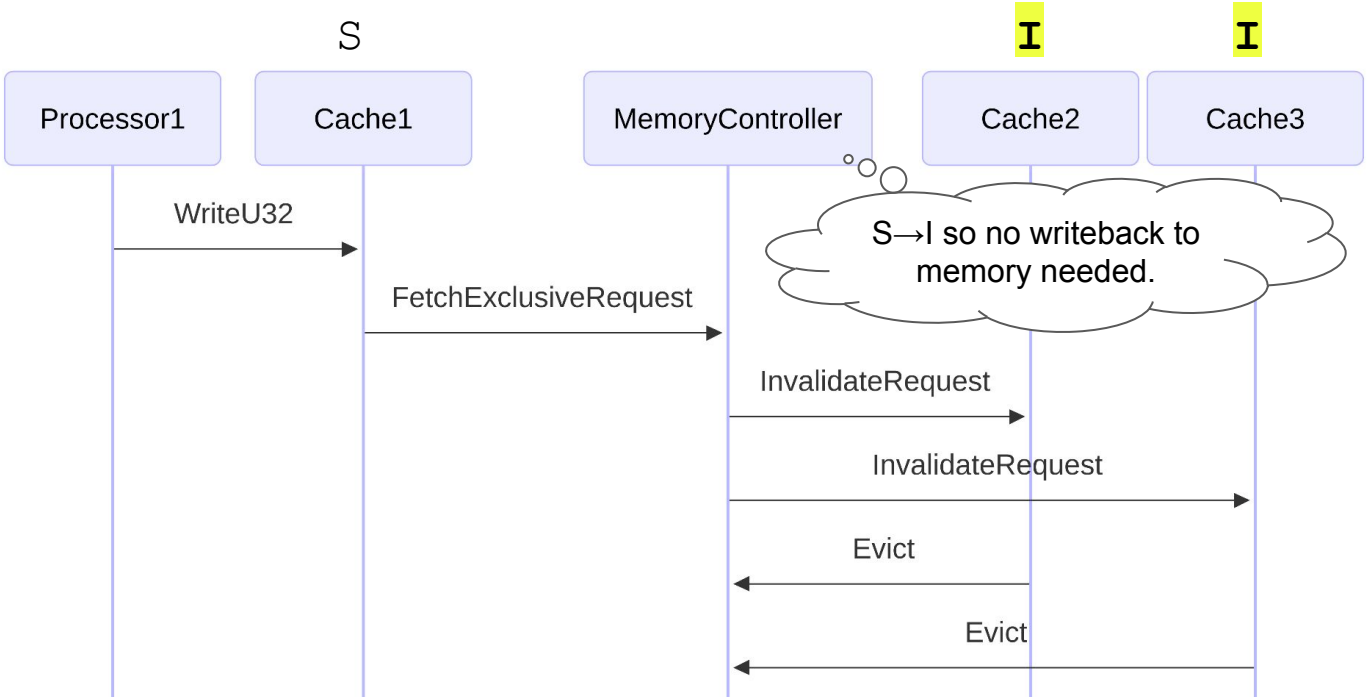
Directory-based MSI write example



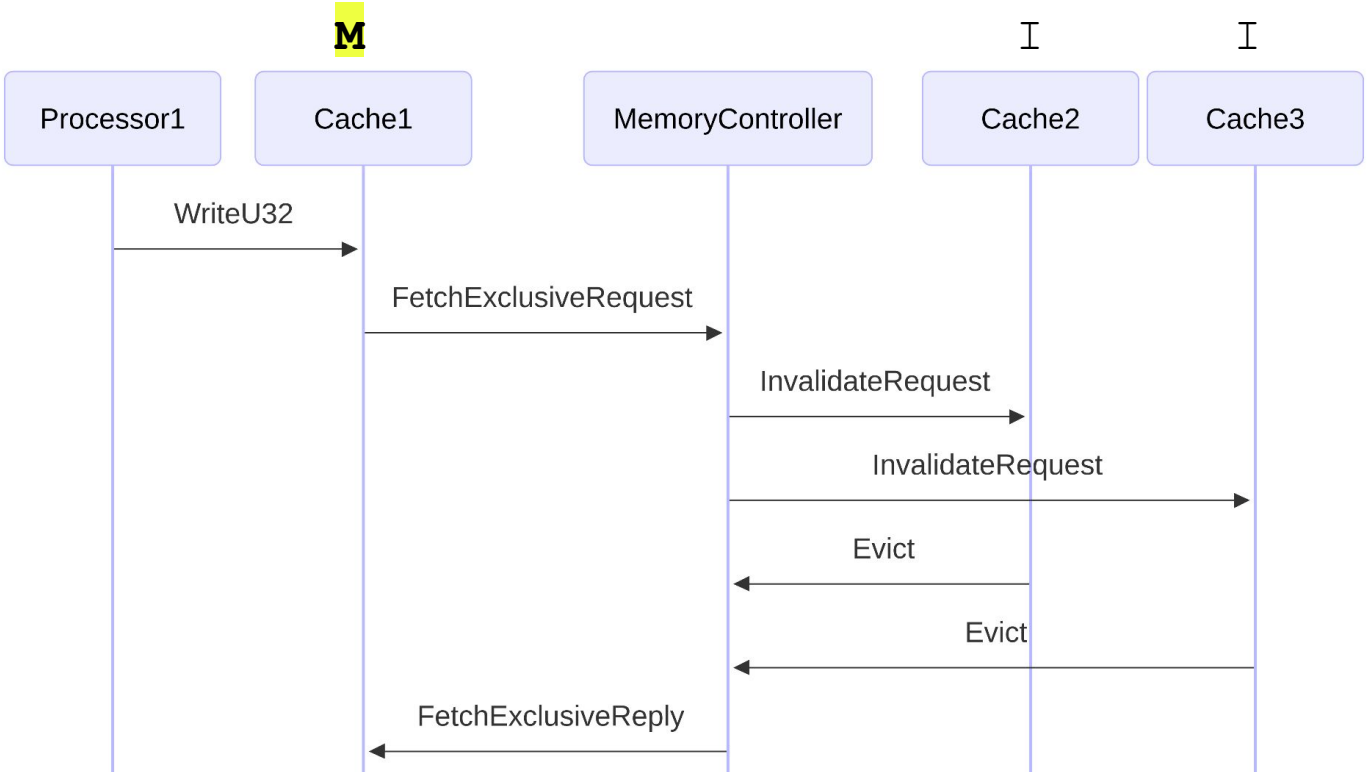
Directory-based MSI write example



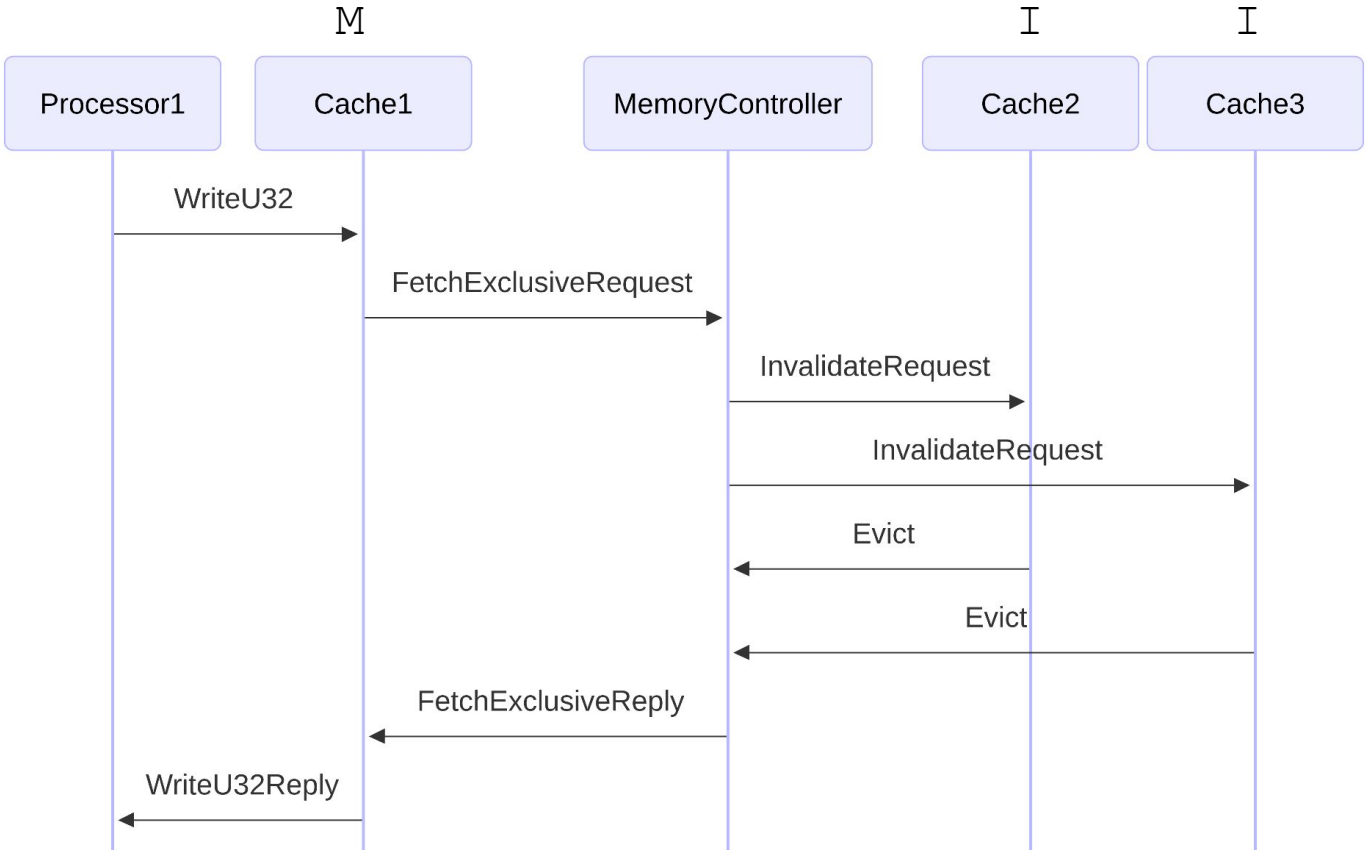
Directory-based MSI write example



Directory-based MSI write example



Directory-based MSI write example



Snooping

- Simpler hardware.
- Lower latency.

Directory-based

- Scales better with hundreds of processors
- No shared bus arbitration

- Typical computers use snooping with **snoop filters**.
- **Snoop filters** are incomplete mini-directories along the shared bus.
- They may filter out some messages that are irrelevant to caches on their bus segment.
- They reduce shared bus traffic, and power usage.

Extending MSI

	Dirty	Clean
Exclusive	M	
Shared		S

- We can add more possible line states.
- Goal: create new protocols with more states to optimize out unnecessary requests and memory accesses.

Extending MSI

	Dirty	Clean
Exclusive	**M**	**E**
Shared		**S**

- We can add more possible line states.
- Goal: create new protocols with more states to optimize out unnecessary requests and memory accesses.
- **E (exclusive)**. This line **matches** memory, and is in **only 1** cache. Read-only.

Extending MSI

	Dirty	Clean
Exclusive	M	E
Shared	O	S

- We can add more possible line states.
- Goal: create new protocols with more states to optimize out unnecessary requests and memory accesses.
- **E (exclusive)**. This line **matches** memory, and is in **only 1** cache. Read-only.
- **O (owned)**. This line **differs** from memory, and **many** caches have copies of it in the **S** state. Read-only.

MESI

	Dirty	Clean
Exclusive	M	E
Shared	O	S

	M	E	S	I
M	❌	❌	❌	✅
E	❌	❌	❌	✅
S	❌	❌	✅	✅
I	✅	✅	✅	✅

In MSI, a processor might:

- Read (**I**→**S**), then immediately write (**S**→**M**).
- Cache must send two separate requests.

Improvement: MESI (Modified, Exclusive, Shared, Invalid)

- On cache read, if no other caches have this line, transitions **I**→**E**.
- On write, silently upgrades **E**→**M** without sending any messages.
- On evict (**E**→**I**), no writeback is needed.
- When another cache requests a read, downgrades **E**→**S** and replies with the line (cache-to-cache transfer).

MOSI

	Dirty	Clean
Exclusive	M	E
Shared	O	S

	M	O	S	I
M	❌	❌	❌	✅
O	❌	❌	✅	✅
S	❌	✅	✅	✅
I	✅	✅	✅	✅

Scenario in MSI:

- Processor 1 writes (**I**→**M**), and then processor 2 reads it (**I**→**S**).
- Cache 1 must perform slow writeback to memory (**M**→**S**).

Improvement: MOSI (Modified, Owned, Shared, Invalid)

- When cache 2 reads (**I**→**S**), cache 1 directly transfers its line to it, without writeback to memory. Cache 1 also downgrades its line (**M**→**O**).
- Cache with **O** is responsible for eventual writeback to memory (on **O**→**I**).

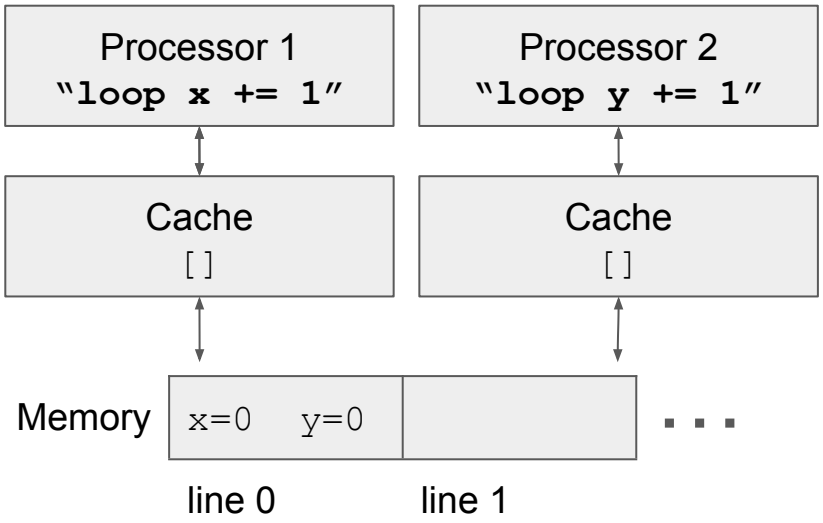
MOESI and MESIF

Goal: further remove unnecessary requests and memory accesses.

- AMD: MOESI (MOSI + MESI).
- Intel: MESIF
  - “Forward” designates one cache to do cache-to-cache transfers to other caches that request a line. Used to avoid bus collisions by preventing multiple volunteers.

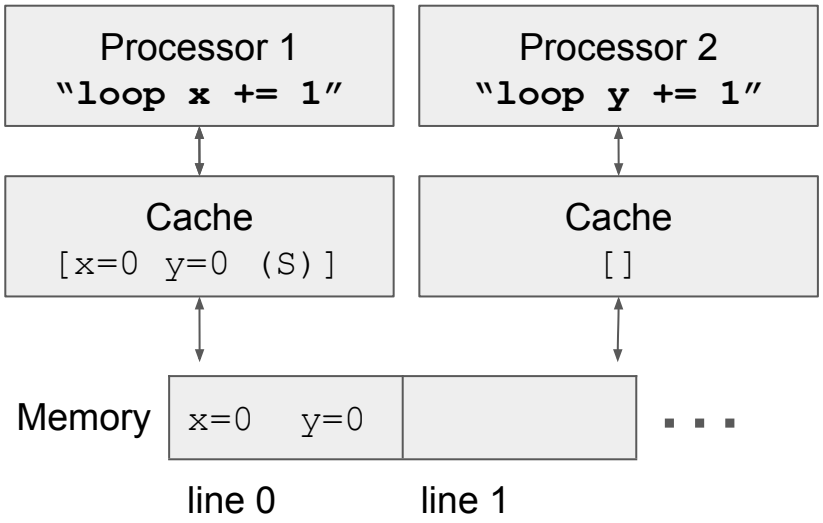
False sharing

- **True sharing**: processors access the same data.
- **False sharing**: processors access different data which happens to be in the same cache line.

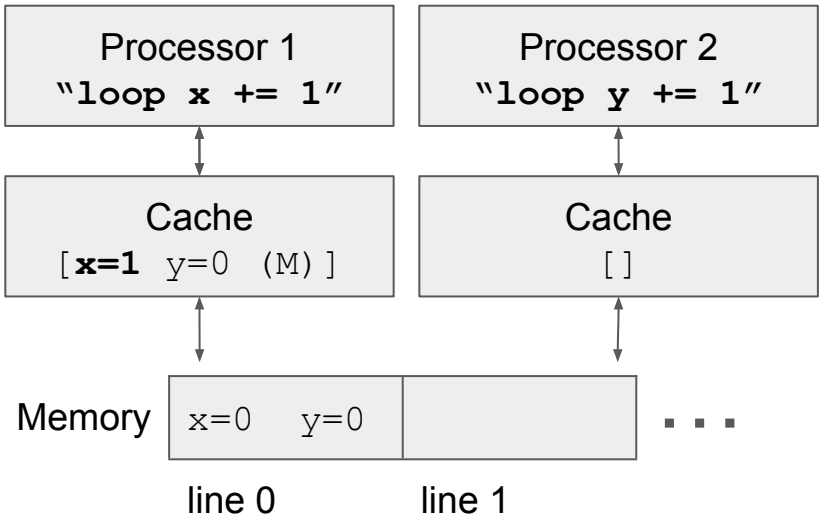


For this example, assume MSI.

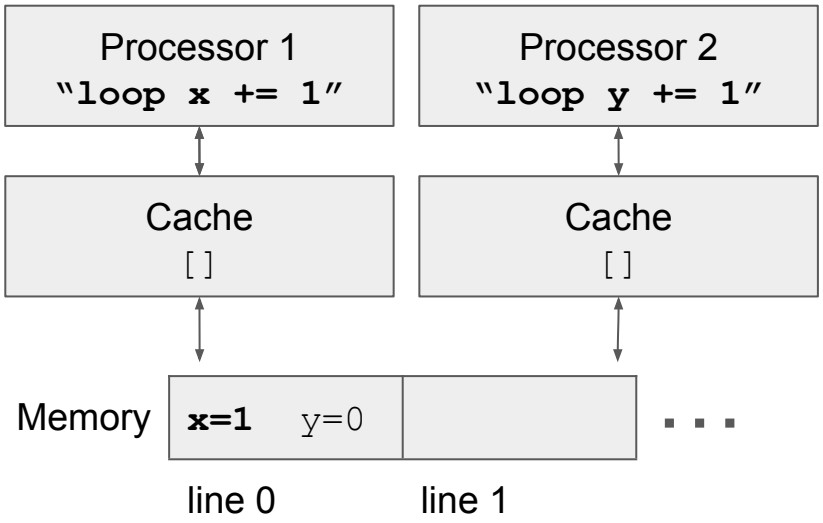
False sharing



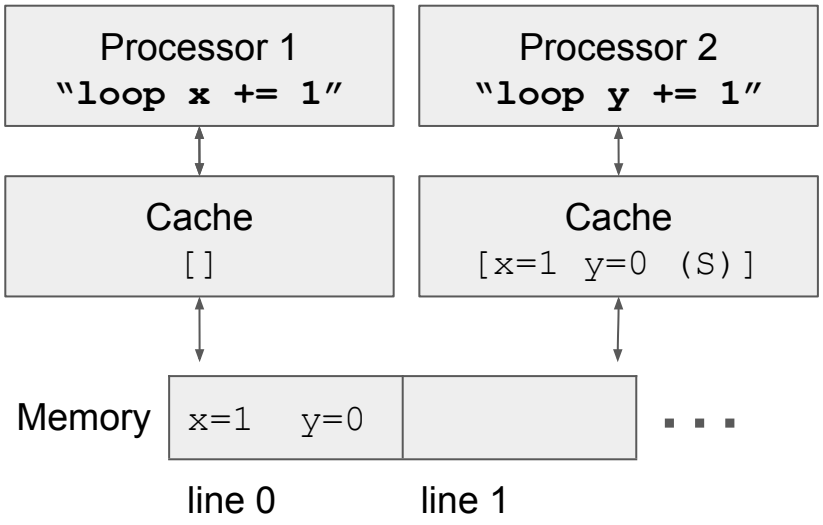
False sharing



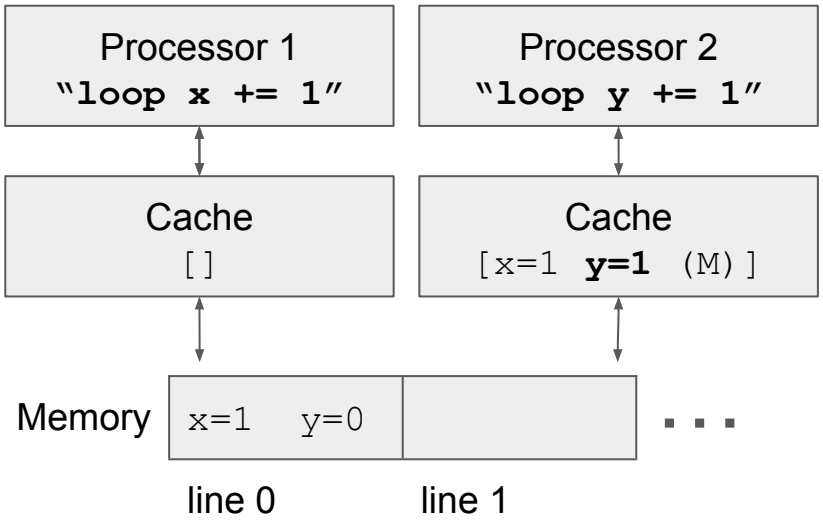
False sharing



False sharing

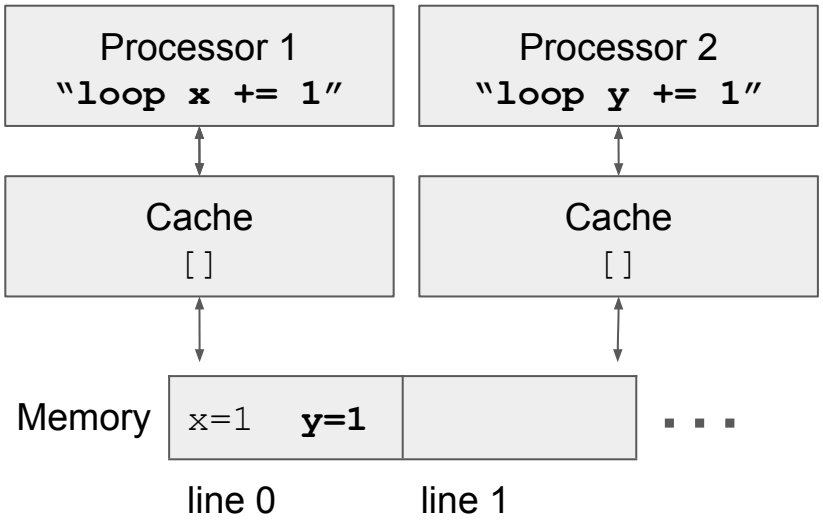


False sharing



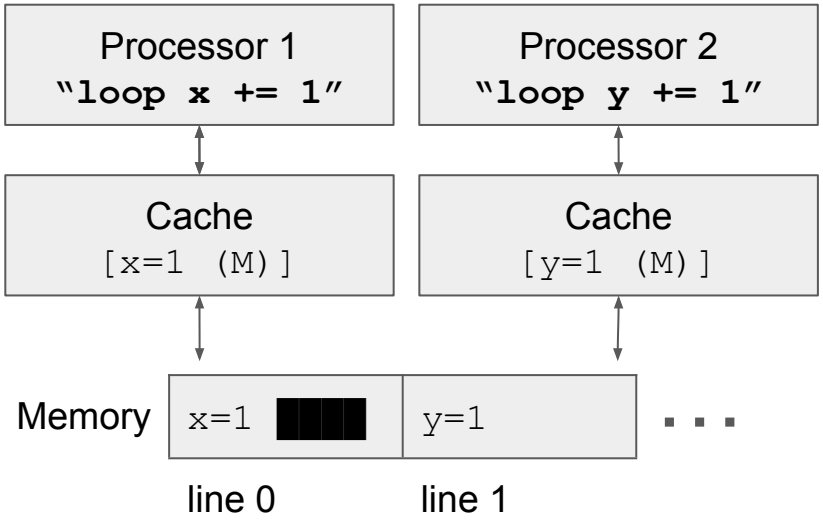
False sharing

- Cache coherence upheld, but performance suffers.
- We can turn these memory writebacks into cache-to-cache transfers. But they're still slow.



False sharing

- Remove by adding padding.
- Separates variables into different lines.

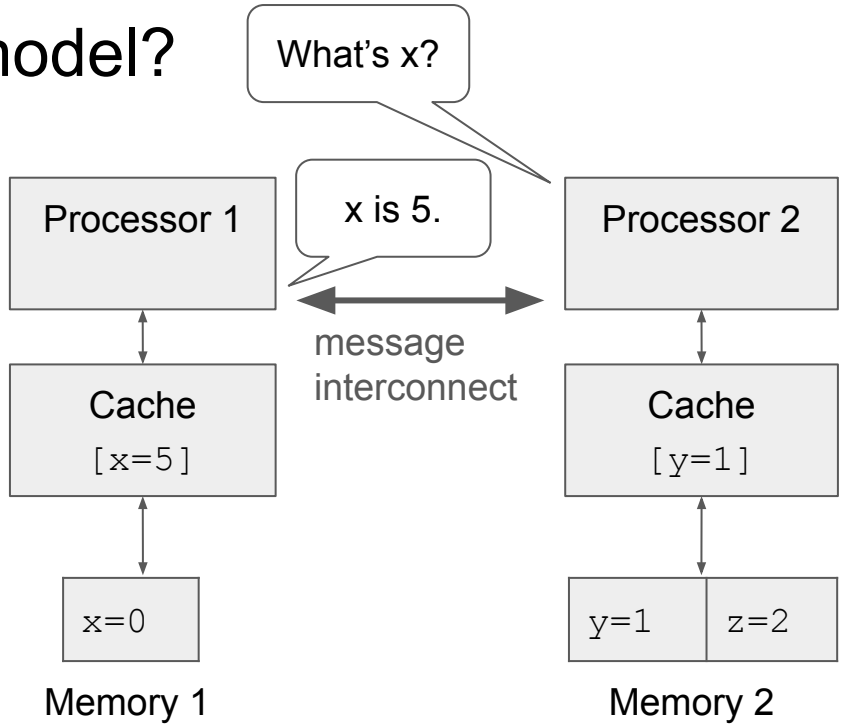


Alternative: message passing model?

- Most hardware follows a **shared memory model.**
- In the **message passing model** cache coherence wouldn’t be a problem because:
  - Each core has its own memory.
  - They communicate by passing messages.

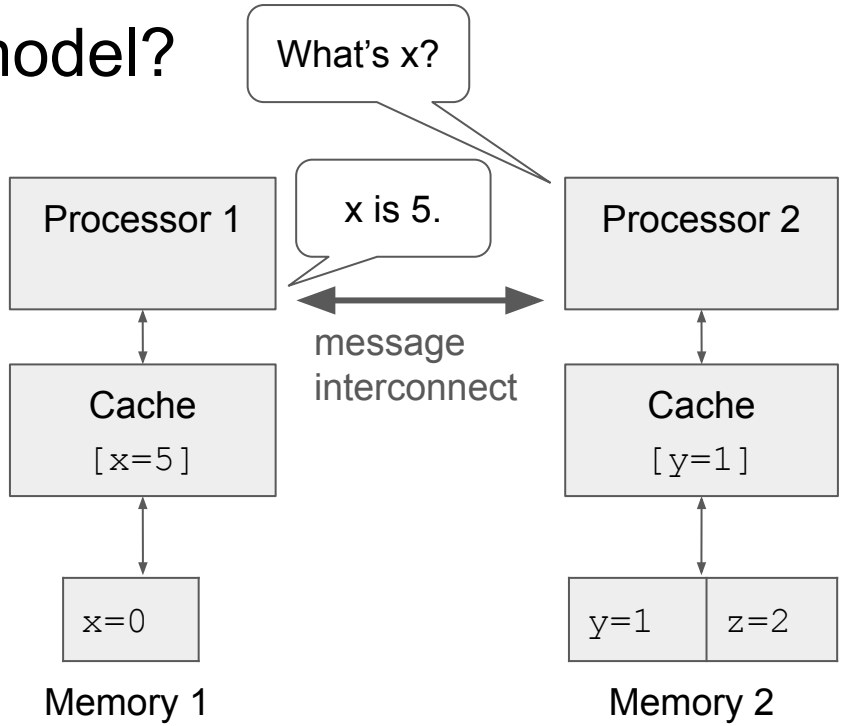
Alternative: message passing model?

- Most hardware follows a **shared memory model.**
- In the **message passing model** cache coherence wouldn't be a problem because:
  - Each core has its own memory.
  - They communicate by passing messages.



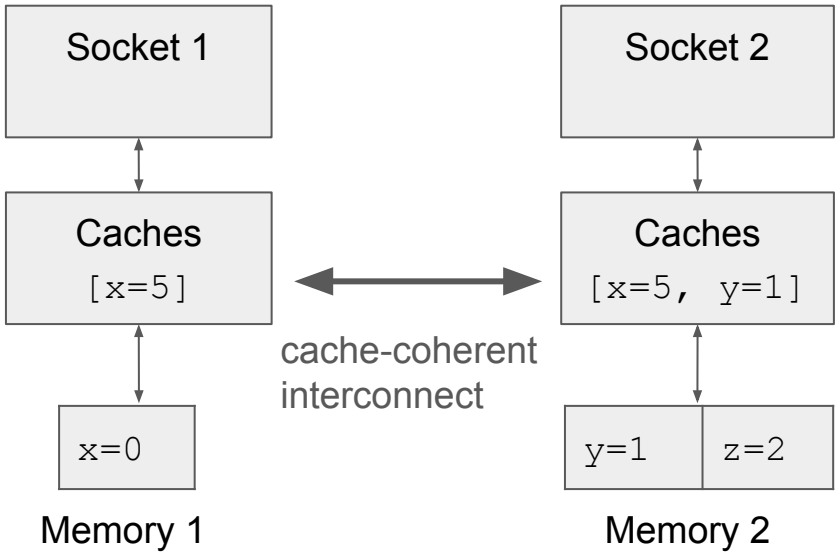
Alternative: message passing model?

- **Message passing** *is* used, but at higher levels: channels, http requests.
- But in hardware, the **shared memory model** is used for ease of programming.
  - A thread retains access to the same memory when it's scheduled on different cores.



Aside: NUMA (non-uniform memory access)

- A high-end server architecture.
- Each CPU socket has its own local memory, for performance.
- A cache-coherent interconnect provides a **shared memory address space**.
- Can access memory of other sockets, but it's much slower.



Quiz on Gradescope (11:14-11:20)

Quiz on Gradescope (11:14-11:20)

A. We can eliminate deadlocks by ordering our locks and never acquiring a lock with a lower or equal ID than one we have already acquired.
B. If a type T is Sync, then a reference to that type (&T) is Send.
C. In Rust, model checking for the purpose of detecting buggy race conditions requires modeling instruction-level interleavings.
D. The purpose of a RefCell is to implement interior mutability by checking borrow rules at runtime.
E. Arc::clone makes a deep copy of the pointed-to data.